Laporan Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial



Optimasi Penempatan Kendaraan pada Geladak Kapal Menggunakan Algoritma Local Search

Kelompok: G35 - NamaTim2

Anggota:

18224041	Irghi Satya Priangga
18224039	Lutfi Al Pasha
18224105	Kyan Saktya Diraya

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

Jalan Ganesa No. 10 Bandung

Daftar Isi

Daftar Isi **2**
Deskripsi Persoalan **3**
Pembahasan **4**
1. Implementasi Algoritma 4
1.1 Implementasi Hill-Climbing 4
a. Penjelasan Kode Algoritma 4
b. Fungsi dari modul lain yang dipanggil 7
1.2 Implementasi Simulated Annealing 7
a. Penjelasan Kode Algoritma 7
b. Fungsi dari modul lain yang dipanggil 10
1.3 Implementasi Genetic Algorithm 11
a. Penjelasan Kode Algoritma 11
b. Fungsi dari modul lain yang dipanggil 15
2. Hasil Eksperimen 16
2.1 Hasil Eksperimen Hill-Climbing 16
2.2 Hasil Eksperimen Simulated Annealing 16
2.3 Hasil Eksperimen Genetic Algorithm 17
3. Analisis 19
3.1 Ringkasan Hasil 19
3.2 Kedekatan dengan Global Optima 20
3.3 Perbandingan Hasil Antar Algoritma 20
3.4 Perbandingan Durasi Pencarian 21
3.5 Konsistensi Hasil Akhir 21
3.6 Pengaruh Banyak Iterasi Jumlah Populasi pada Genetic Algorithm 22
3.7 Keterbatasan 22
Kesimpulan dan Saran **23**
Kesimpulan 23
Saran 23
Pembagian Tugas **25**
Referensi **26**
Surat Pernyataan Penggunaan AI **27**

Deskripsi Persoalan

Anda adalah seorang *Forward Deployed Engineer* (FDE) di PT Intim Line yang mengoperasikan kapal kargo pengangkut kendaraan. Perusahaan baru saja memperoleh kontrak pengiriman dari beberapa dealer kendaraan sekaligus. Pada setiap jadwal keberangkatan, perusahaan menerima daftar kendaraan dengan dimensi, berat, tarif pengiriman, dan deadline yang berbeda-beda.

Selama ini, penempatan kendaraan pada geladak kapal dilakukan secara manual. Akibatnya, ruang geladak sering tidak termanfaatkan dengan baik dan keputusan pemilihan letak kendaraan belum tentu menghasilkan keuntungan yang optimal.

Tim operasional meminta Anda membangun sistem pendukung keputusan untuk menentukan kendaraan yang akan diangkut serta posisi dan orientasi setiap kendaraan pada geladak. Sistem tersebut harus dapat mengevaluasi berbagai kemungkinan konfigurasi, mempertimbangkan fungsi-fungsi objektif yang ditentukan, dan menghasilkan solusi penataan yang optimal.

Pembahasan

1. Implementasi Algoritma

Penjelasan implementasi ketiga algoritma (Hill Climbing, Simulated Annealing, dan Genetic Algorithm) terhadap masalah Optimasi Penempatan Kendaraan pada Geladak Kapal Menggunakan Algoritma Local Search.

1.1 Implementasi Hill-Climbing

Varian yang dipakai adalah Stochastic Hill Climbing. Pada tiap iterasi, algoritma mengambil satu neighbor acak, lalu berpindah ke neighbor tersebut hanya jika nilainya lebih baik dari state saat ini.Alasan dipilih Stochastic Hill Climbing karena ruang neighbor besar, jauh lebih murah daripada Steepest Ascent Hill Climbing yang harus mengevaluasi (is_valid) setiap iterasi, Stochastic Hill Climbing hanya mengevaluasi sekali random state. Alasan selanjutnya adalah karena semua komponen yang dibutuhkan sudah ada tanpa tambahan (random_initial_state dan random_neighbor) dibanding hill climbing yang lain butuh all_neighbor untuk divalidasi.Alasan terakhir adalah karena cocok dengan masalah kapal dan kendaraan, move di masalah ini banyak menghasilkan state yang tidak valid, maka Stochastic Hill Climbing cocok karena state tidak valid langsung dilewat tanpa dilanjut, biaya lebih murah.

Tetapi Stochastic Hill Climbing memiliki beberapa kelemahan. Yang pertama mudah terjebak di local optimum, tidak ada mekanisme keluar, kecuali kebetulan neighbor acak lebih baik. Yang kedua dapat terjebak di plateau, neighbor bernilai sama ditolak (> bukan >=), padahal di masalah ini banyak state sama nilainya (misalnya memutar kendaraan). Yang ketiga hasilnya tidak deterministik, perlu beberapa run untuk memenuhi kualitas. Yang terakhir yaitu max_no_improve bisa berhenti terlalu cepat atau terlalu lama.

Berikut merupakan penjelasan algoritma Stochastic Hill Climbing

a. Penjelasan Kode Algoritma

Fungsi utama Hill Climbing dijelaskan sebagai berikut.

1. Parameternya:
   - ship dan vehicles: data kapal dan daftar kendaraan.
   - max_iterations (default 1000): batas maksimum iterasi.
   - max_no_improve (default 200): pencarian berhenti jika sebanyak ini iterasi berturut-turut tidak menghasilkan perbaikan.
2. Alur:
   - Buat state awal acak yang valid dengan random_initial_state, lalu simpan salinannya sebagai initial agar state awal tidak ikut berubah.

- Hitung nilai objective state awal dan simpan ke history (indeks 0 adalah nilai sebelum iterasi pertama).
- Pada tiap iterasi, ambil satu neighbor acak yang valid dengan random_neighbor.
- Jika tidak ada neighbor valid (None), iterasi dihitung sebagai iterasi tanpa perbaikan, dan state tetap.
- Jika nilai neighbor lebih besar dari nilai saat ini, neighbor diterima dan penghitung no_improve direset ke 0.
- Jika nilai neighbor sama atau lebih buruk, neighbor ditolak (nilai yang sama ditolak, jadi tidak ada sideways move) dan no_improve bertambah.
- Nilai objective saat ini dicatat ke history pada setiap iterasi.
- Pencarian berhenti saat max_iterations tercapai atau no_improve mencapai max_no_improve.

3. Hasil:
   Algorithm, Initial State, Final State, Initial Value, Final Value, History, Duration (detik), dan Iterations.

```
#File hill_climbing.py untuk tes Stochastic Hill Climbing pada
kasus kendaraan dan kapal
#Mengembalikan format hasil yang terdiri dari:
#1. Algorithm: Nama algoritma
#2. Initial State: state awal
#3. Final State: state akhir
#4. Initial Value: nilai objective state awal (dalam satuan juta
rupiah)
#5. Final Value: nilai objective state akhir (dalam satuan juta
rupiah)
#6. History: nilai objective tiap iterasi untuk plot
#7. Duration: durasi pencarian (dalam detik)
#8. Iterations: banyak iterasi hingga pencarian berhenti
import time
from objective import objective
from neighbors import random_initial_state, random_neighbor

#Fungsi stochastic_hil_climbing untuk tes kasus dengan algoritma
stochastic hill climbing
def stochastic_hill_climbing(ship, vehicles, max_iterations=1000,
max_no_improve=200):
    start = time.perf_counter()
```

```
    current = random_initial_state(ship, vehicles)  #cari random
state awal yang valid
    initial = current.copy()                        #kerjakan di
salinan agar state awal tidak berubah
    current_value = objective(current)              #nilai
objective dari state awal
    history = [current_value]                       #nilai
objective sebelum iterasi pertama (indeks = 0)
    no_improve = 0
    iterations = 0

    while iterations < max_iterations and no_improve <
max_no_improve:
        iterations += 1
        neighbor = random_neighbor(current)         #cari random
neighbor yang ingin dibandingkan dengan state awal
        if neighbor is None:                        #jika tidak
ada neighbor yang valid, dihitung sebagai iterasi tanpa perbaikan
(perpindahan)
            no_improve += 1
            history.append(current_value)
            continue
        neighbor_value = objective(neighbor)
        if neighbor_value > current_value:          #stocahstic
hill climbing akan menerima neighbor jika nilainya lebih baik
(kalau nilainya sama ditolak)
            current = neighbor
            current_value = neighbor_value
            no_improve = 0
        else:                                       #jika neighbor
nilainya sama atau lebih buruk, tetap di current state dan
dihitung sebagai iterasi tanpa perbaikan (perpindahan)
            no_improve += 1
        history.append(current_value)

    duration = time.perf_counter() - start
    return{
        "Algorithm": "Stochastic Hill Climbing",
```

```
        "Initial State": initial,
        "Final State": current,
        "Initial Value": history[0],
        "Final Value": current_value,
        "History": history,
        "Duration": duration,
        "Iterations": iterations,
    }
```

b. Fungsi dari modul lain yang dipanggil

Fungsi	Modul
random_initial_state	neighbors.py
random_neighbor	neighbors.py
objective	objective.py
State.copy	models.py

1.2 Implementasi Simulated Annealing

Simulated Annealing (SA) mengambil satu neighbor acak pada tiap iterasi. Neighbor yang lebih baik selalu diterima. Neighbor yang sama atau lebih buruk tetap bisa diterima dengan probabilitas e^(delta/T), sehingga algoritma bisa keluar dari local optima. Suhu T turun tiap iterasi, jadi penerimaan state yang lebih buruk makin jarang seiring waktu.

a. Penjelasan Kode Algoritma

Fungsi utama Simulated Annealing.

1. Parameternya:
   - ship dan vehicles: data kapal dan daftar kendaraan.
   - initial_temp (default 100.0): suhu awal T0.
   - cooling_rate (default 0.995): faktor pendinginan geometrik, T = T * cooling_rate.
   - min_temp (default 0.01): pencarian berhenti jika suhu turun di bawah nilai ini.
   - max_iterations (default 10000): batas maksimum iterasi.

   Karena kasusnya maksimasi, delta = neighbor_value - current_value.
2. Alurnya:
   - Buat state awal acak yang valid dengan random_initial_state, simpan salinannya sebagai initial, dan catat nilai objective-nya ke history.

- Pada tiap iterasi, ambil satu neighbor acak yang valid dengan random_neighbor.
- Jika tidak ada neighbor valid (None), state tetap, iterasi dihitung sebagai stuck, probabilitas dicatat 0.0, lalu suhu diturunkan.
- Jika delta > 0 (neighbor lebih baik), neighbor selalu diterima dengan probabilitas 1.0.
- Jika delta <= 0, neighbor diterima dengan probabilitas e^(delta/T). Jika ditolak, stuck_count bertambah.
- Probabilitas dicatat ke prob_history dan nilai objective saat ini dicatat ke history pada setiap iterasi.
- Suhu diturunkan: temp *= cooling_rate.
- Pencarian berhenti saat max_iterations tercapai atau suhu mencapai min_temp.

3. Hasilnya
   Algorithm, Initial State, Final State, Initial Value, Final Value, History, Duration, Iterations, Prob History, Stuck Count, dan Params (dict berisi initial_temp, cooling_rate, min_temp, dan max_iterations).

```
#File simulated_annealing.py, algoritma Simulated Annealing untuk
memaksimalkan objective (total shipping fee)
#Format hasil (key dictionary) disamakan dengan hill_climbing.py +
metrik khusus SA (Prob History, Stuck Count)
import math
import random
import time
from objective import objective
from neighbors import random_initial_state, random_neighbor

#Fungsi simulated_annealing
#Parameter:
#   initial_temp     : suhu awal T0
#   cooling_rate     : faktor pendinginan geometrik (T = T *
cooling_rate), 0 < cooling_rate < 1
#   min_temp         : suhu minimum, pencarian berhenti jika T <
min_temp
#   max_iterations   : batas maksimum iterasi
#Karena kasusnya maksimasi, delta = neighbor_value - current_value
#   delta > 0  -> neighbor selalu diterima
#   delta <= 0 -> neighbor diterima dengan probabilitas
```

```
e^(delta/T)
def simulated_annealing(ship, vehicles, initial_temp=100.0,
cooling_rate=0.995,
                        min_temp=0.01, max_iterations=10000):
    start = time.perf_counter()

    current = random_initial_state(ship, vehicles)  #state awal
random yang valid
    initial = current.copy()                        #salinan agar
state awal tidak berubah
    current_value = objective(current)
    history = [current_value]                       #nilai
objective sebelum iterasi pertama (indeks = 0)
    prob_history = []                               #nilai
e^(delta/T) tiap iterasi (1.0 jika neighbor lebih baik / tidak ada
neighbor)
    stuck_count = 0                                 #frekuensi
'stuck' di local optima (neighbor lebih buruk/sama ditolak)
    temp = initial_temp
    iterations = 0

    while iterations < max_iterations and temp > min_temp:
        iterations += 1
        neighbor = random_neighbor(current)
        if neighbor is None:                        #tidak ada
neighbor valid -> tetap di current state, dihitung stuck
            stuck_count += 1
            prob_history.append(0.0)
            history.append(current_value)
            temp *= cooling_rate
            continue

        neighbor_value = objective(neighbor)
        delta = neighbor_value - current_value
        if delta > 0:                               #neighbor
lebih baik -> selalu diterima
            prob = 1.0
            accept = True
        else:                                       #neighbor
```

```
sama/lebih buruk -> diterima dengan probabilitas e^(delta/T)
            prob = math.exp(delta / temp)
            accept = random.random() < prob
            if not accept:
                stuck_count += 1

        if accept:
            current = neighbor
            current_value = neighbor_value
        prob_history.append(prob)
        history.append(current_value)
        temp *= cooling_rate                          #turunkan suhu

    duration = time.perf_counter() - start
    return {
        "Algorithm": "Simulated Annealing",
        "Initial State": initial,
        "Final State": current,
        "Initial Value": history[0],
        "Final Value": current_value,
        "History": history,
        "Duration": duration,
        "Iterations": iterations,
        "Prob History": prob_history,
        "Stuck Count": stuck_count,
        "Params": {
            "initial_temp": initial_temp,
            "cooling_rate": cooling_rate,
            "min_temp": min_temp,
            "max_iterations": max_iterations,
        },
    }
```

b. Fungsi dari modul lain yang dipanggil

Fungsi	Modul
random_initial_state	neighbors.py
random_neighbor	neighbors.py

objective	objective.py
State.copy	models.py

1.3 Implementasi Genetic Algorithm

Genetic Algorithm (GA) merupakan algoritma optimasi yang menggunakan sekumpulan kandidat solusi (population) dan mekanisme evolusi untuk mencari solusi dengan nilai fitness terbaik. Pada permasalahan penempatan kendaraan di geladak kapal, satu individu direpresentasikan sebagai sebuah State yang berisi posisi dan orientasi seluruh kendaraan. Nilai fitness individu dihitung menggunakan fungsi objective, dengan tujuan memaksimalkan total tarif pengiriman tanpa melanggar batasan penempatan kendaraan.

Genetic Algorithm memiliki kelebihan berupa kemampuan mengeksplorasi banyak kandidat solusi melalui populasi, sehingga pencarian tidak hanya bergantung pada satu state awal. Mekanisme crossover memungkinkan penggabungan karakteristik dari dua parent, sedangkan mutasi memberikan variasi tambahan pada kandidat solusi. Selain itu, penggunaan elitism memastikan individu terbaik dari generasi sebelumnya tetap dipertahankan.

Namun, Genetic Algorithm juga memiliki beberapa kelemahan. Pertama, biaya komputasi dapat meningkat seiring bertambahnya ukuran populasi dan jumlah generasi karena semakin banyak individu yang harus dievaluasi. Kedua, hasilnya tidak deterministik karena proses pembentukan populasi, pemilihan parent, crossover, dan mutasi melibatkan pengacakan. Ketiga, populasi yang terlalu kecil dapat mengurangi keragaman solusi sehingga pencarian kurang efektif. Terakhir, mekanisme repair pada proses crossover dapat mengeluarkan kendaraan yang menyebabkan penempatan tidak valid. Akibatnya, child yang dihasilkan bisa memiliki nilai fitness lebih rendah daripada parent-nya.
Algoritma ini menggunakan beberapa mekanisme utama, yaitu pembentukan populasi awal, tournament selection, crossover, mutasi, dan elitism.

Berikut merupakan penjelasan algoritma Genetic Algorithm

a. Penjelasan Kode Algoritma

Fungsi utama Genetic Algorithm dijelaskan sebagai berikut.

1. Parameternya:
   - ship dan vehicles: data kapal dan daftar kendaraan yang akan ditempatkan.
   - pop_size: jumlah individu dalam populasi.
   - iterations: jumlah generasi yang dijalankan oleh algoritma.

- mutation_rate (default 0.2): probabilitas terjadinya mutasi pada setiap individu yang diproses.

2. Fungsi pendukung:
   - init_population: membuat populasi awal yang terdiri dari pop_size state acak yang valid menggunakan random_initial_state. Setiap individu awal memenuhi constraint, yaitu kendaraan tidak saling tumpang tindih, berada dalam batas geladak, dan tidak melebihi kapasitas kapal.
   - tournament_selection: memilih satu parent melalui mekanisme turnamen. Sebanyak \(k\) individu dipilih secara acak, kemudian individu dengan nilai fitness tertinggi menjadi pemenang. Nilai default \(k\) adalah 3, dengan jumlah peserta dibatasi agar tidak melebihi ukuran populasi.
   - crossover: menghasilkan satu child dari dua parent. Child dimulai dari state kosong, kemudian kendaraan diproses dalam urutan acak. Untuk setiap kendaraan, salah satu parent dipilih secara acak untuk menentukan posisi dan orientasinya. Jika penempatan kendaraan menyebabkan state tidak valid, kendaraan tersebut dikembalikan ke None, yang berarti kendaraan tidak diangkut. Dengan mekanisme perbaikan (repair) ini, child tetap valid.
   - mutate: mengubah state dengan probabilitas sesuai mutation_rate. Jika mutasi terjadi, state diganti dengan satu neighbor yang dihasilkan oleh random_neighbor, seperti rotate, relocate, atau swap. Jika mutasi tidak terjadi atau tidak ditemukan neighbor yang valid, state dikembalikan tanpa perubahan.
3. Alurnya:
   - Buat populasi awal menggunakan init_population, lalu hitung nilai fitness seluruh individu menggunakan objective.
   - Cari individu dengan nilai fitness tertinggi dalam populasi awal dan simpan salinannya sebagai initial.
   - Pada setiap generasi, catat nilai fitness maksimum dan rata-rata populasi ke dalam history_max dan history_avg.
   - Salin individu terbaik dari populasi saat ini ke populasi baru melalui mekanisme elitism. Dengan demikian, individu terbaik tidak hilang akibat proses reproduksi.
   - Isi sisa populasi baru dengan memilih dua parent menggunakan tournament_selection.
   - Lakukan crossover terhadap kedua parent untuk menghasilkan child, kemudian jalankan mutate terhadap child tersebut.
   - Ulangi proses pemilihan parent, crossover, dan mutasi hingga populasi baru mencapai ukuran pop_size.

- Ganti populasi lama dengan populasi baru, kemudian hitung ulang nilai fitness seluruh individu.
- Setelah seluruh generasi selesai, pilih individu dengan nilai fitness tertinggi sebagai state akhir.

4. Hasilnya

Algorithm, Initial State, Final State, History Max, History Average, Duration, Iterations, dan Population Size.

init_population

```
# Fungsi init_population untuk membuat populasi awal berisi state acak yang valid
def init_population(ship, vehicles, pop_size):
    return [random_initial_state(ship, vehicles) for _ in range(pop_size)]
```

tournament_selection

```
# Fungsi tournament_selection untuk memilih satu parent dari populasi
def tournament_selection(population, fitness=None, k=3):
    if fitness is None:
        fitness = [objective(s) for s in population]
    k=min(k, len(population))
    participants = random.sample(range(len(population)), k)
    winner = max(participants, key=lambda i: fitness[i])
    return population[winner]
```

mutate

```
# Fungsi mutate untuk mengubah state sedikit dengan peluang mutation_rate
def mutate(state, mutation_rate=0.2):
    if random.random() < mutation_rate:
        neighbor = random_neighbor(state)
        if neighbor is not None:
            return neighbor
    return state
```

crossover

```
# Fungsi crossover untuk membuat satu child dari dua parent
# Tiap kendaraan diambil dari salah satu parent, lalu diperbaiki
(repair):
# kendaraan yang membuat state tidak valid dikembalikan ke None
(tidak diangkut)
def crossover(parent1, parent2):
    child = State(
        parent1.ship, copy.deepcopy(parent1.vehicles),
        {vid: None for vid in parent1.vehicles},
    )
    order = list(child.vehicles.keys())
    random.shuffle(order)
    for vid in order:
        parent = random.choice([parent1, parent2])
        if parent.positions[vid] is None:
            continue
        child.vehicles[vid].orientation =
parent.vehicles[vid].orientation
        child.positions[vid] = parent.positions[vid]
        if not is_valid(child):
            child.positions[vid] = None
    return child
```

genetic_algorithm

```
# Fungsi genetic_algorithm sebagai fungsi utama
def genetic_algorithm(ship, vehicles, pop_size, iterations,
mutation_rate=0.2):
    start = time.perf_counter()

    population = init_population(ship, vehicles, pop_size)
    fitness = [objective(s) for s in population]
    best_index = max(range(len(population)), key=lambda i:
fitness[i])
    initial = population[best_index].copy()
    history_max = []
     history_avg = []
```

```
    for _ in range(iterations):
        history_max.append(max(fitness))
        history_avg.append(sum(fitness) / len(fitness))

        best_index = max(range(len(population)), key=lambda i:
fitness[i])
        new_population = [population[best_index].copy()]
        while len(new_population) < pop_size:
            a = tournament_selection(population, fitness)
            b = tournament_selection(population, fitness)
            child = crossover(a, b)
            child = mutate(child, mutation_rate)
            new_population.append(child)
        population = new_population
        fitness = [objective(s) for s in population]

    best_index = max(range(len(population)), key=lambda i:
fitness[i])
    final = population[best_index]
    duration = time.perf_counter() - start
    return {
        "Algorithm": "Genetic Algorithm",
         "Initial State": initial,
        "Final State": final,
        "History Max": history_max,
        "History Average": history_avg,
        "Duration": duration,
        "Iterations": iterations,
        "Population Size": pop_size,
    }
```

b. Fungsi dari modul lain yang dipanggil

Fungsi	Modul
random_initial_state	neighbors.py
random_neighbor	neighbors.py
objective	objective.py

is_valid	objective.py
State.copy	models.py

2. Hasil Eksperimen

2.1 Hasil Eksperimen Hill-Climbing

Eksperimen dijalankan 3 kali dengan `python src/run_hc.py`. Datanya memakai satu kapal 10x20 (kapasitas 500) dan 14 kendaraan dari `data.py`, sama dengan algoritma lain. Parameter HC: `max_iterations=1000`, `max_no_improve=200`. State awal dibuat acak di setiap run.

Run	Nilai awal	Nilai akhir	Kenaikan	Iterasi	Durasi (detik)
1	403	415	+12	393	0.1969
2	366	403	+37	239	0.1022
3	378	453	+75	305	0.2471
Rata-rata	382.3	423.6	+41.3	312.3	0.182

Nilai dalam juta rupiah. Gambar yang perlu disisipkan per run (semua di folder `output/`):

Keterangan	File
State awal	`hc_run{1,2,3}_initial.png`
State akhir	`hc_run{1,2,3}_final.png`
Plot objective terhadap iterasi	`hc_run{1,2,3}_curve.png`

2.2 Hasil Eksperimen Simulated Annealing

Eksperimen dijalankan 3 kali dengan `python src/run_sa.py`. Datanya memakai satu kapal 10x20 (kapasitas 500) dan 14 kendaraan dari `data.py`, sama dengan algoritma lain. Parameter SA: `initial_temp=100.0`, `cooling_rate=0.998`, `min_temp=0.01`, `max_iterations=10000`. State awal dibuat acak di setiap run.

Run	Nilai awal	Nilai akhir	Kenaikan	Iterasi	Durasi (detik)	Stuck
1	385	563	+178	4601	2,6436	632
2	380	553	+173	4601	2,8435	195
3	293	528	+235	4601	2,4021	499
Rata-rata	352,7	548,0	+195,3	4601	2,63	442

Nilai dalam juta rupiah. Gambar yang perlu disisipkan per run (semua di folder `output/`):

Keterangan	File
State awal	`sa_run{1,2,3}_initial.png`
State akhir	`sa_run{1,2,3}_final.png`
Plot objective terhadap iterasi	`sa_run{1,2,3}_curve.png`
Plot e^(ΔE/T) terhadap iterasi	`sa_run{1,2,3}_prob.png`

2.3 Hasil Eksperimen Genetic Algorithm

Eksperimen dijalankan 3 kali untuk tiap konfigurasi dengan python src/run_ga.py. Datanya memakai satu kapal 10x20 (kapasitas 500) dan 14 kendaraan dari data.py, sama dengan algoritma lain. Parameter GA: mutation_rate=0.2, tournament k=3, elitism 1 individu. Ada dua eksperimen. Eksperimen 1 memakai populasi 30 sebagai kontrol dengan iterasi 50, 100, dan 200. Eksperimen 2 memakai iterasi 100 sebagai kontrol dengan populasi 10, 30, dan 60. State awal adalah individu terbaik pada populasi awal, bukan individu acak biasa.

Eksperimen 1: populasi tetap (30), iterasi divariasikan

Iterasi	Run	Nilai awal	Nilai akhir	Kenaikan	Durasi
50	1	433	470	+37	0,3994
50	2	468	478	+10	0,4465
50	3	433	468	+35	0,4535

50	**Rata-rata**	444,7	472,0	+27,3	0,43
100	1	473	503	+30	0,8027
100	2	461	498	+37	0,8486
100	3	478	478	0	0,8312
100	**Rata-rata**	470,7	493,0	+22,3	0,83
200	1	478	513	+35	1,6880
200	2	458	478	+20	1,6913
200	3	453	528	+75	1,6977
200	**Rata-rata**	463,0	506,3	+43,3	1,69

Eksperimen 2: iterasi tetap (100), populasi divariasikan

Populasi	Run	Nilai awal	Nilai akhir	Kenaikan	Durasi
10	1	461	473	+12	0,2817
10	2	428	436	+8	0,2387
10	3	391	403	+12	0,2261
10	**Rata-rata**	426,7	437,3	+10,7	0,25
30	1	446	458	+12	0,8239
30	2	473	473	0	0,7180
30	3	466	496	+30	1,0105
30	**Rata-rata**	461,7	475,7	+14,0	0,85
60	1	478	478	0	1,6506
60	2	451	523	+72	1,8672
60	3	453	478	+25	1,6414
60	**Rata-rata**	460,7	493,0	+32,3	1,72

Nilai dalam juta rupiah. Konfigurasi populasi 30 dan iterasi 100 muncul di kedua eksperimen sebagai dua rangkaian run terpisah, sehingga hasilnya berbeda. Gambar yang perlu disisipkan per run (semua di folder output/):

Keterangan	File
State awal	ga_{exp}_pop{pop}_iter{iter}_run{1,2,3}_initial.png
State akhir	ga_{exp}_pop{pop}_iter{iter}_run{1,2,3}_final.png
Plot nilai maksimum dan rata-rata populasi terhadap iterasi	ga_{exp}_pop{pop}_iter{iter}_run{1,2,3}_curve.png

3. Analisis

Semua angka diambil dari hasil eksperimen pada folder output (hc_summary.json, sa_summary.json, ga_summary.json). Satuan nilai objective adalah juta rupiah.

3.1 Ringkasan Hasil

Algoritma / Konfigurasi	Rata-rata Nilai Awal	Rata-rata Nilai Akhir	Nilai Akhir Tiap Run	Rata-rata Durasi (detik)
Hill Climbing (stochastic)	382,3	**423,7**	415, 403, 453	0,18
Simulated Annealing	352,7	**548,0**	563, 553, 528	2,63
GA, populasi 30, iterasi 50	444,7	**472,0**	470, 478, 468	0,43
GA, populasi 30, iterasi 100	470,7	**493,0**	503, 498, 478	0,83
GA, populasi 30, iterasi 200	463,0	**506,3**	513, 478, 528	1,69
GA, populasi 10, iterasi 100	426,7	**437,3**	473, 436, 403	0,25
GA, populasi 30, iterasi 100 (eksperimen 2)	461,7	**475,7**	458, 473, 496	0,85
GA, populasi 60, iterasi 100	460,7	**493,0**	478, 523, 478	1,72

Catatan: pada Genetic Algorithm, nilai awal adalah individu terbaik pada populasi awal, bukan satu state acak. Karena itu nilai awal GA lebih tinggi daripada Hill Climbing dan Simulated Annealing, dan kenaikannya terlihat lebih kecil.

3.2 Kedekatan dengan Global Optima

Global optima yang sebenarnya tidak diketahui, sehingga dipakai batas atas sebagai pembanding. Jumlah berat semua kendaraan (545) melebihi kapasitas kapal (500), sehingga setidaknya satu kendaraan harus tertinggal. Pilihan terbaik adalah melepas V08 (tarif 38, berat 45), dengan total berat tepat 500 dan nilai 588. Batas atas ini belum tentu bisa dicapai karena seluruh kendaraan harus muat di geladak 10x20 tanpa tumpang tindih. Total luas kendaraan yang dipilih (193) hampir memenuhi luas geladak (200).

Algoritma	Rata-rata Akhir	Persen dari 588	Terbaik	Persen dari 588
Hill Climbing	423,7	72,1%	453	77,0%
Genetic Algorithm (terbaik: populasi 30, iterasi 200)	506,3	86,1%	528	89,8%
Simulated Annealing	548,0	93,2%	563	95,7%

- Hill Climbing hanya menerima neighbor yang lebih baik, sehingga cepat terjebak di local optima. Pencarian berhenti setelah sekitar 240 sampai 390 iterasi karena 200 iterasi berturut-turut tidak menghasilkan perbaikan.
- Simulated Annealing menerima langkah yang lebih buruk dengan peluang e^(ΔE/T). Pada awal pencarian suhu tinggi sehingga nilai sempat turun (pada run 1 sekitar 300 sampai 415) sebelum naik kembali. Hal ini membantu keluar dari local optima. Setelah suhu turun, perilakunya mendekati hill climbing dan nilai stabil di sekitar 563 pada run 1.
- Genetic Algorithm mencari dengan banyak individu sekaligus, tetapi pada kasus ini crossover mudah menghasilkan child yang tidak valid. Perbaikan dilakukan dengan melepas kendaraan yang bentrok, sehingga child kemungkinan sering lebih buruk daripada parent. Dugaan ini perlu dikonfirmasi pada kurva GA.

3.3 Perbandingan Hasil Antar Algoritma

- Simulated Annealing menghasilkan nilai akhir tertinggi, dengan rata-rata 548 dibandingkan 423,7 (Hill Climbing) dan 506,3 (GA terbaik).
- Run SA terburuk (528) sama dengan run GA terbaik pada konfigurasi populasi 30 iterasi 200 (528), dan berada di atas run Hill Climbing terbaik (453).
- GA lebih baik daripada Hill Climbing pada hampir semua konfigurasi. Satu pengecualian: GA populasi 10 iterasi 100 (rata-rata 437,3) hanya sedikit di atas Hill Climbing (423,7).

3.4 Perbandingan Durasi Pencarian

Algoritma	Durasi	Keterangan
Hill Climbing	0,10 sampai 0,25 detik	Tercepat, berhenti lebih awal
Genetic Algorithm	0,23 sampai 1,87 detik	Naik kira-kira linear terhadap populasi x iterasi
Simulated Annealing	2,40 sampai 2,84 detik	Terlama, 4601 iterasi sampai suhu mencapai batas bawah

- Biaya per langkah ketiga algoritma kurang lebih sama, sekitar 0,3 sampai 0,6 milidetik per neighbor atau child. Yang membedakan durasi adalah jumlah langkah yang dilakukan.
- Dengan anggaran waktu yang mirip, Simulated Annealing tetap lebih baik. GA populasi 60 iterasi 100 (1,72 detik) memiliki rata-rata 493, sedangkan SA (2,63 detik) rata-rata 548.
- Hill Climbing dan GA populasi 10 iterasi 100 memiliki durasi yang hampir sama (0,18 dan 0,25 detik) dengan hasil yang hampir sama (423,7 dan 437,3).

3.5 Konsistensi Hasil Akhir

Algoritma / Konfigurasi	Rentang Nilai Akhir Antar Run	Selisih
Hill Climbing	403 sampai 453	50
Simulated Annealing	528 sampai 563	35
GA, populasi 30, iterasi 50	468 sampai 478	10
GA, populasi 30, iterasi 100	478 sampai 503	25
GA, populasi 30, iterasi 200	478 sampai 528	50
GA, populasi 10, iterasi 100	403 sampai 473	70
GA, populasi 60, iterasi 100	478 sampai 523	45

- Semua algoritma memiliki variasi antar run karena state awal dan langkah pencarian bersifat acak. Setiap konfigurasi hanya dijalankan 3 kali, sehingga hasilnya belum cukup untuk kesimpulan statistik yang kuat.
- Konfigurasi GA populasi 30 iterasi 100 muncul di dua eksperimen dengan hasil berbeda (rata-rata 493 dan 475,7). Selisih sekitar 17 ini menunjukkan besarnya variasi acak, sehingga perbedaan antar konfigurasi GA yang lebih kecil dari itu belum tentu bermakna.
- Simulated Annealing relatif konsisten dan semua run berada di atas 528. Jumlah stuck-nya sangat bervariasi (195 sampai 632) tetapi tidak berkaitan dengan nilai akhir: run 2 paling sedikit stuck (195) dengan nilai 553, sedangkan run 1 paling banyak stuck (632) dengan nilai 563.

3.6 Pengaruh Banyak Iterasi Jumlah Populasi pada Genetic Algorithm

Eksperimen 1: jumlah populasi 30 sebagai kontrol, banyak iterasi divariasikan.

Iterasi	Rata-rata Nilai Akhir	Kenaikan	Durasi (detik)
50	472,0	-	0,43
100	493,0	+21,0	0,83
200	506,3	+13,3	1,69

Eksperimen 2: banyak iterasi 100 sebagai kontrol, jumlah populasi divariasikan.

Populasi	Rata-rata Nilai Akhir	Kenaikan	Durasi (detik)
10	437,3	-	0,25
30	475,7	+38,4	0,85
60	493,0	+17,3	1,72

- Menambah iterasi atau populasi meningkatkan hasil rata-rata, tetapi keuntungannya makin kecil, sedangkan durasi naik sekitar dua kali lipat setiap kali.
- Populasi kecil (10) paling merugikan karena keragaman individu rendah dan hasilnya paling dekat dengan Hill Climbing. Peningkatan dari populasi 10 ke 30 (+38,4) lebih besar daripada dari 30 ke 60 (+17,3).
- Karena menggunakan elitism, nilai maksimum populasi tidak turun antar generasi. Hasil akhir bisa sama dengan nilai awal bila tidak ada child yang lebih baik, seperti pada salah satu run dengan nilai 478.

3.7 Keterbatasan

- Setiap konfigurasi hanya dijalankan 3 kali dan tanpa seed tetap.
- Jumlah langkah tiap algoritma berbeda (Hill Climbing berhenti dini, Simulated Annealing mengikuti jadwal suhu, GA mengikuti populasi x iterasi), sehingga perbandingan durasi dan hasil tidak sepenuhnya setara.
- Parameter Simulated Annealing (initial_temp, cooling_rate) dan Hill Climbing (max_no_improve) hanya diuji dengan satu setelan.
- Batas atas 588 hanya berdasarkan kapasitas berat, bukan geometri geladak.

Kesimpulan dan Saran

Kesimpulan

1. Persoalan penempatan kendaraan pada geladak kapal berhasil dimodelkan sebagai masalah local search. State berupa posisi dan orientasi tiap kendaraan (atau tidak diangkut), neighbor berupa rotate, relocate, dan swap, dan objective berupa total tarif pengiriman yang dimaksimalkan dengan constraint tidak tumpang tindih, berada dalam batas geladak, dan tidak melebihi kapasitas.
2. Simulated Annealing memberikan hasil terbaik pada eksperimen ini, dengan rata-rata nilai akhir 548,0 juta rupiah (93,2% dari batas atas 588) dan nilai terbaik 563. Genetic Algorithm terbaik (populasi 30, iterasi 200) mencapai rata-rata 506,3, dan Hill Climbing mencapai rata-rata 423,7.
3. Hill Climbing paling cepat (rata-rata 0,18 detik) tetapi cepat terjebak di local optima. Simulated Annealing paling lama (rata-rata 2,63 detik) tetapi paling dekat dengan nilai optimum. Genetic Algorithm berada di antara keduanya, dengan durasi yang naik mengikuti jumlah populasi dan iterasi.
4. Pada Genetic Algorithm, menambah jumlah iterasi maupun populasi meningkatkan hasil rata-rata, tetapi dengan peningkatan yang semakin kecil sementara durasi naik sekitar dua kali lipat. Populasi kecil (10) memberikan hasil paling rendah.
5. Hasil tiap algoritma bervariasi antar run karena sifat acaknya. Dengan hanya 3 run per konfigurasi, perbedaan kecil antar konfigurasi belum cukup untuk disimpulkan secara pasti.
6. Untuk persoalan dengan 14 kendaraan pada kapal 10x20 ini, Simulated Annealing paling sesuai bila yang diutamakan adalah kualitas solusi. Hill Climbing sesuai bila waktu sangat terbatas dan hasil kurang optimal dapat diterima.

Saran

1. Jalankan setiap konfigurasi lebih dari 3 kali, dan gunakan seed yang dicatat, agar hasil lebih stabil dan dapat direproduksi serta dibandingkan dengan lebih adil.
2. Pada Simulated Annealing, uji beberapa nilai initial_temp dan cooling_rate, serta skema pendinginan lain (misalnya linear atau logaritmik), karena eksperimen ini hanya memakai satu setelan.
3. Pada Hill Climbing, coba varian lain seperti Sideways Move dan Random Restart, serta ubah max_no_improve. Varian Random Restart berpotensi memperbaiki hasil Hill Climbing yang sering terjebak di local optima.
4. Pada Genetic Algorithm, evaluasi pengaruh mutation_rate, ukuran turnamen, dan cara perbaikan child hasil crossover. Perlu dicek apakah perbaikan yang melepas kendaraan bentrok membuat child sering lebih buruk daripada parent.

5. Samakan anggaran pencarian antar algoritma (misalnya jumlah evaluasi objective yang sama) agar perbandingan hasil dan durasi lebih setara.
6. Uji pada data yang lebih besar atau bervariasi (lebih banyak kendaraan, ukuran kapal berbeda) untuk melihat apakah urutan kualitas antar algoritma tetap sama.
7. Kembangkan lebih lanjut dengan objective function alternatif atau dukungan lebih dari satu kapal, sebagaimana yang disebutkan pada spesifikasi bonus.

Pembagian Tugas

NIM	Nama	Tugas
18224041	Irghi Satya Priangga	1. Membuat file: - src/data.py - src/hill_climbing.py - src/models.py - src/neighbors.py - src/objective.py - src/run_hc.py - src/visualization.py - README.md 2. Membuat repository dan semua folder di dalamnya 3. Merapikan dan menyusun dokumen terutama bagian stochastic hill climbing 4. Menjalankan eksperimen Stochastic Hill Climbing sebanyak 3 kali dan menyimpan hasilnya di output/ 5. Testing keseluruhan kode
18224039	Lutfi Al Pasha	1. Membuat file: - src/genetic_algorithm.py - src/run_ga.py 2. Menyusun dan merapikan dokumen laporan terutama bagian implementasi algoritma dan hasil eksperimen genetic algorithm 3. Menambahkan fungsi plot_ga_curve pada src/visualization.py 4. Menjalankan eksperimen genetic algorithm dan menyimpan hasilnya di output/
18224105	Kyan Saktya Diraya	1. Membuat file: - src/simulated_annealing.py - src/run_sa.py 2. Menambahkan fungsi plot_sa_probability pada src/visualization.py. 3. Menjalankan eksperimen Simulated Annealing sebanyak 3 kali dan menyimpan hasilnya di output/. 4. Menyusun bagian Simulated Annealing pada dokumen laporan.

Referensi

S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2021. [Online]. Available: https://aima.cs.berkeley.edu/

D. E. Goldberg, Genetic Algorithms in Search, Optimization, and Machine Learning. Reading, MA, USA: Addison-Wesley, 1989. Available: https://books.google.com/books?vid=ISBN978-0-201-15767-3

Power Point Materi Local Search Mata Kuliah IF3070 Dasar Intelegensi Artifisial.

M. L. Khodra, “Beyond Classical Search” (Classical vs Local Search, State, Hill-climbing, Simulated Annealing, Genetic Algorithm), YouTube, 2021.

S. Kirkpatrick, C. D. Gelatt, dan M. P. Vecchi, “Optimization by Simulated Annealing,” Science, vol. 220, no. 4598, hlm. 671-680, 1983.

N. Metropolis dkk., “Equation of State Calculations by Fast Computing Machines,” The Journal of Chemical Physics, vol. 21, no. 6, hlm. 1087-1092, 1953.

B. L. Miller dan D. E. Goldberg, “Genetic Algorithms, Tournament Selection, and the Effects of Noise,” Complex Systems, vol. 9, no. 3, hlm. 193-212, 1995.

A. Lodi, S. Martello, dan M. Monaci, “Two-dimensional packing problems: A survey,” European Journal of Operational Research, vol. 141, no. 2, hlm. 241-252, 2002.

J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” Computing in Science & Engineering, vol. 9, no. 3, hlm. 90-95, 2007.

J. H. Holland, Adaptation in Natural and Artificial Systems, University of Michigan Press, 1975.

D. E. Goldberg, Genetic Algorithms in Search, Optimization, and Machine Learning, Addison-Wesley, 1989.

Surat Pernyataan Penggunaan AI

Dengan ini, kami:
Nama/NIM : - Irghi Satya Priangga/18224041
- Lutfi Al Pasha/18224039
- Kyan Saktya Diraya/18224105

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF3070
Nama Mata Kuliah : Dasa Intelegensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1 Optimasi Penempatan Kendaraan pada Geladak Kapal Menggunakan Algoritma Local Search

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas. Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Sedang
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Tinggi
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Irghi Satya Priangga
18224041